Semantics and Optimizations for 2D Graphics

ANONYMOUS AUTHOR(S)

In recent years, significant advances have been made in numerical debugging and static analysis tools. Recent debugging tools significantly reduce overhead using double-double oracles, while recent static analysis tools accurately bound rounding errors using condition numbers. However, these new techniques also have downsides: double-double oracles can miss floating-point errors due to correlated errors, while condition number formalisms cannot cleanly handle over- and underflow. We show that combining both techniques avoids the downsides of each and implement this combination in EasterEgg. EasterEgg uses double-double computations but not as an oracle; instead, it detects erroneous operations using condition numbers, which minimizes overhead while achieving precision and recall similar to more expensive methods. EasterEgg also introduces a separate oracle to detect harmful over- and underflows more accurately; we implement this oracle in a new performant number representation. As a result, EasterEgg achieves a precision of 80.0% and a recall of 96.1% on a collection of 500 difficult numeric benchmarks. An ablation study shows that EasterEgg combines the strengths of both double-double oracles and condition numbers: it is more accurate than double-double oracles yet dramatically faster than methods based on arbitrary-precision condition number computations.

ACM Reference Format:
Anonymous Author(s). 2026. Semantics and Optimizations for 2D Graphics. 1, 1 (March 2026), 18 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

2 Skia

Pavel: make sure we dont reference colors

Skia is a popular library for 2D graphics. Skia's basic APIs create drawable 2D "surfaces" and "layers", draw shapes to those surfaces and layers, blend them together, and apply effects. These APIs are accessed by calling methods on a Skia `SkCanvas` object; however, internally, these `SkCanvas` operations just append commands to a command buffer, which we can take to be a stand-alone program. The program is executed when Skia's `flush` method is called. This section introduces the language's basic operations.

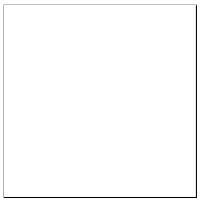
Bhargav: I need to get codex to read the skia source to make sure this is how it works

2.1 Drawing and Painting

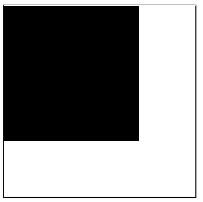
The `drawRect` command is used to draw rectangles:[1]

[1]All code snippets in this section elide the `SkCanvas` object and method-call syntax to maintain focus on the Skia command language itself. We likewise drop the `Sk` prefix on most types.

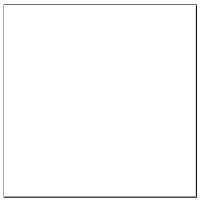
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn



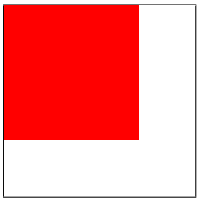
```
SkPaint p;
canvas->drawRect(rect1, p);
```



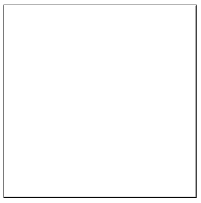
Here the layout is meant to evoke a Hoare triple: the left square shows the (empty) state of the canvas before the `drawRect` command, while the right square shows the state of the canvas afterwards. The `drawRect` command needs the bounds of the rectangle that needs to be drawn. `drawRect` also takes a `SkPaint` as an argument. A `SkPaint` describes *how* a geometry is drawn, for example how to *fill* the geometry. The default fill for a `SkPaint` is the color black, but we can also change it to another color.



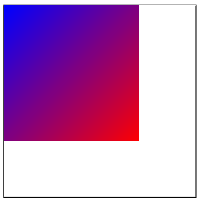
```
SkPaint p;
p.setColor(SK_ColorRED);
canvas->drawRect(rect1, p);
```



It's also possible to specify a non-constant fill, like a gradient:



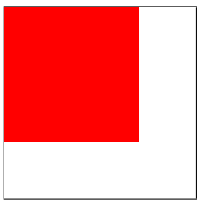
```
SkPaint p;
p.setShader(
  SkShaders::LinearGradient(...)
);
canvas->drawRect(rect1, p);
```



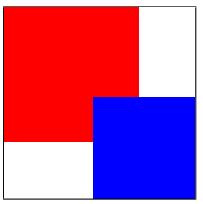
Skia also supports complex fills like images and shaders. Formally, filling is a stage of Skia's drawing pipeline that assigns a color to every point in the shape.

2.2 Blending

Things get a little more interesting once there is more than one shape being drawn. Suppose we draw a second, overlapping, blue square:

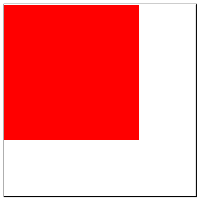


```
p.setColor(SK_ColorBLUE);
canvas->drawRect(rect2, p);
```

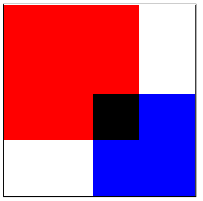


In Skia, the process of drawing this second shape involves, conceptually, two stages. First, as before, every pixel in `Rect2` is assigned a fill color, here a solid blue. Second, this assigned fill

color is *blended* with the existing color for that pixel. The default *blend mode*, called `SrcOver`, just chooses the new color,[2] but a program can select a different blend modes instead. For example, the `Multiply` blend mode combines both colors being blended:



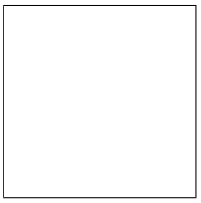
```
p.setColor(SK_ColorBLUE);
p.setBlendMode(
  SkBlendMode::kMultiply
);
canvas->drawRect(rect2, p);
```



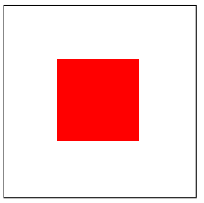
Now, in the intersection of the two rectangles, the initial red color is blended with the newly-drawn blue fill, which in the `Multiply` blend mode results in black. Formally, blending is another stage in the Skia drawing pipeline, which comes after filling and combines the fill color assigned to each pixel with the pre-existing color of that pixel.

2.3 Clipping

So far, `drawRect` has been applying the fill to the entirety of the shape being drawn. But, sometimes, we need more granual control over the drawing bounds. Skia provides a dedicated `clipRect` command to restrict or *clip* shapes to rectangles.[3] One uses `clipRect` like this:

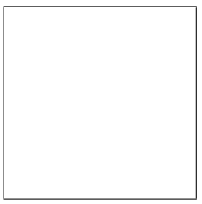


```
canvas->clipRect(c_rect);
canvas->drawRect(rect1, redPaint);
```

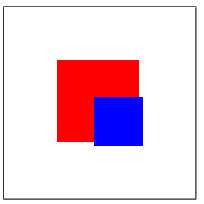


Note the unusual order of operations here. The `clipRect` command is run first, and it does not modify the image at all. Instead, it modifies the shape drawn by *later* draw operations. Formally, clipping is yet another stage of the Skia drawing pipeline; it comes before filling and modifies the shape being drawn.

Semantically, `clipRect` modifies the *clip state*, which stored in the canvas object and affects drawing commands. This means that a single `clipRect` operation can affect multiple `drawRect` operations:

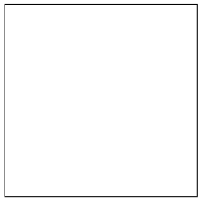


```
canvas->clipRect(c_rect);
canvas->drawRect(rect1, redPaint);
canvas->drawRect(rect2, bluePaint);
```

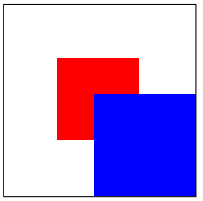


[2] In reality, the blending is more complicated and depends on the opacity or *alpha* of the new color, which can matter for anti-aliasing, but in our examples all the colors are opaque and we ignore anti-aliasing as a simplification.
[3] Skia allows clipping with arbitrary shapes, but the GPU back-end has fast paths for rectangles and rounded-rectangles, reducing them to scissor tests

Naturally, one might want to clip only a subset of drawing commands. Luckily, Skia maintains a *stack* of clip states; `clipRect` affects the top of the stack, which is used by drawing commands, while `save` and `restore` commands push and pop from the stack. `save` and `restore` act like brackets, restricting the effect of `clip` to the enclosed commands:



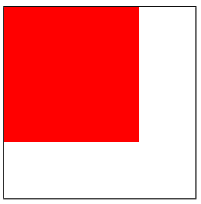
```
canvas->save();
canvas->clipRect(c_rect);
canvas->drawRect(rect1, redPaint);
canvas->restore();
canvas->drawRect(rect2, bluePaint);
```



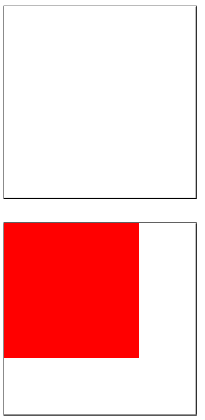
2.4 Layers

Pavel: I thought this was the most abrupt and unmotivated transition. Not the end of the world.

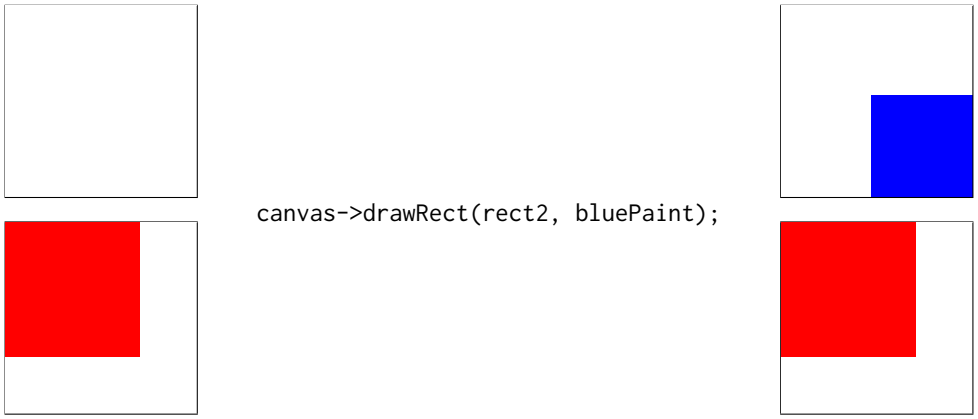
Much like `save` and `restore` can save clip state, there is also a `saveLayer` and `restore` command that saves and restores the drawing surface itself. Conceptually, `saveLayer` allocates a new, empty drawing surface.



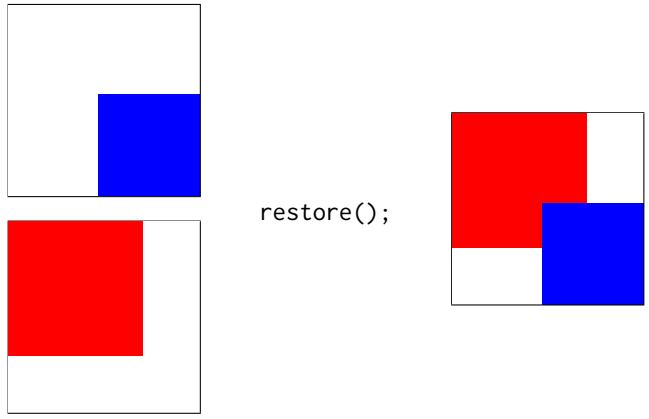
```
canvas->saveLayer();
```



All drawing commands until the corresponding `restore` draw to this new surface.



Then, when the `restore` command it reached, the whole resulting drawing surface is blended back into the original drawing surface.



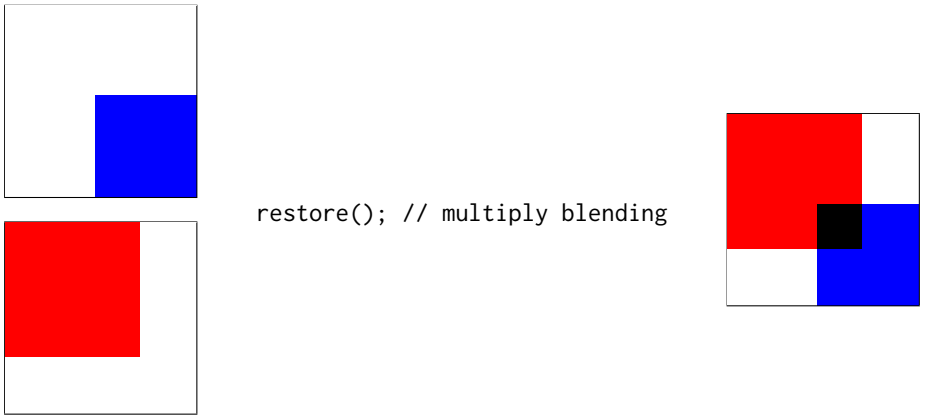
Semantically, you can think of `SaveLayer` commands as re-associating the order of blend operations. In the `Save` example, the order of operations is

$$\text{blend}(\text{blend}(\text{empty}, \text{red}), \text{blue}),$$

While in the `SaveLayer` example the order of operations is

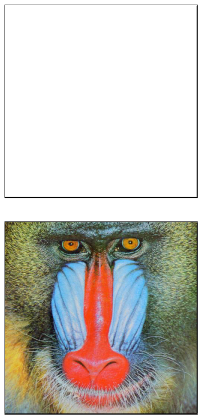
$$\text{blend}(\text{blend}(\text{empty}, \text{red}), \text{blend}(\text{empty}, \text{blue})).$$

With `SrcOver` blending, which is associative, this order of operations is not important, but with other blend modes it becomes significant. For example, we reepat the previous snippet, but pass a `Multiply` blend mode to the `saveLayer` command.

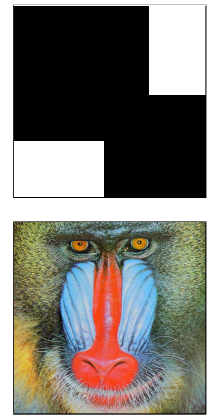


2.4.1 Masking. To further motivate the need for `saveLayers`, consider masking again. In Section 2.3 we saw how we can use `clipRect` to mask images. This works when we only need to mask by a rectangle. But masking with complicated shapes is hard with just clipping. Instead of trying to clip directly with a complicated shape, we can instead draw the clip shape in stages on a seperate layer and mask using that shape.

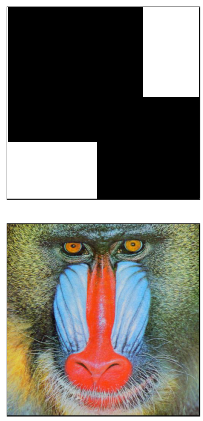
We now draw the mask shape on the new layer.



```
canvas->drawRect(rect2, bluePaint);
```



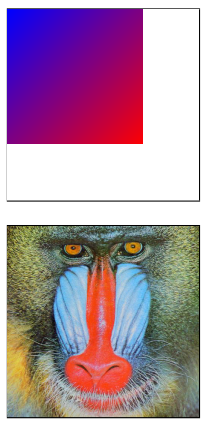
`DstIn` emulates masking through blending; When we `DstIn` one pixel over another, an opaque top pixel keeps the bottom pixel, and a transparent top pixel removes it.



```
restore(); // multiply blending
```

masked_mandrill.pdf

`DstIn` blending is also far more powerful than just clipping, because it also allows masking with gradients, allowing us to create more complicated images that just clipping would not allow us to. For example, here is the previous image, now masked with a radial gradient.



```
restore(); // multiply blending
```

grad_mandrill.pdf

Pavel: I think we require a special Transparent color.

$$
\begin{array}{lcl}
\text{Image} & : & \text{Point} \to \text{Color} \\
\text{Shape} & : & \text{Point} \to \text{Bool} \\
\text{Blend} & : & \text{Color} \times \text{Color} \to \text{Color} \\
\text{Filter} & : & \text{Color} \to \text{Color}
\end{array}
$$

Fig. 1. Our abstract model of Images and Shapes is built from abstract types Pixels and Colors.

```
Solid(Color) | LinearGradient(...) ...                 : Imag
Rect(...) | RRect(...) | Path(...) | Text(...)         : Shap
Id | Luma ...                                          : Filte
SrcOut | DstIn | ...                                   : Blen
```

Fig. 2. Layer provides a simple functional description of Skia images.

2.5 Equivalences

In **??** we saw how complex masks can be done with ease using a `saveLayer` with `DstIn` blending. But what if just a rectangle mask is needed? Our `saveLayer` and `DstIn` program would still work. But, it is clearly less performant than just doing a clip; We have to allocate a whole new layer, just to do something Skia alreayd has a fast path for! Here, we have two different skia programs that produce the same image, but one us clearly a better choice than the other.

```
restore(); // multiply blending

  restore(); // second program
```

`simple_mandrill.pdf`

Therefore it should be possible to replace a skia program with a faster one, given they have indentical results. In Section 3 we develop a semantics, which allows proving that two Skia programs have identical results, including verifying general-purpose rewrite rules. Section 4 then uses these rewrites to build a general-purpose Skia optimizer that rewrites programs to reduce the use of multiple layers. Section 5 then shows that this compiler results in dramatically faster rendering on a wide range of real-world Skia programs.

3 Semantics

This paper's semantics of Skia consists of three components **??**. The bottom-most layer is an abstract model of images, shapes, blends, and filters defined in terms of abstract types Point and Color, which can be instantiated to model as much of the actual Skia API as necessary. The middle layer is functional operations `Draw` and `BlendLayer` and an abstract type Paint, which denote to our abstract model. The top layer is an imperative language, μSkia, whose operational semantics provides a model of Skia. This section describes all three layers and then examines 4 rewrite rules proven valid using the semantics.

3.1 Abstract Model

Traditional programming language semantics typically do not have to define their abstract model, since it is typically based on commonly-known types such as pairs, functions, and integers. In the graphics domain, the core types are not so well established, so the abstract model must be explicitly defined. The abstract model of our semantics is shown in Figure 1; it is based around

$$[\![\texttt{Empty()}]\!](\text{pt}) = \text{Transparent}$$

$$[\![\texttt{BlendLayer}(\texttt{l}_{\texttt{bot}},\ \texttt{l}_{\texttt{top}},\ \texttt{p})]\!](\text{pt}) = \texttt{blend}_{\texttt{p}}\left([\![\texttt{l}_{\texttt{bot}}]\!](\text{pt}),\ \texttt{filter}_{\texttt{p}}\left([\![\texttt{l}_{\texttt{top}}]\!](\text{pt})\right)\right)$$

$$[\![\texttt{Draw}(\texttt{l},\ \texttt{s},\ \texttt{p},\ \texttt{c})]\!](\text{pt}) = \texttt{blend}_{\texttt{p}}\left([\![\texttt{l}(\text{pt})]\!],\ \texttt{filter}_{\texttt{p}}\left(\begin{cases}\texttt{fill}_{\texttt{p}}(\text{pt}) & \text{pt} \in \text{s} \cap \text{c} \\ \text{Transparent} & \text{otherwise}\end{cases}\right)\right)$$

Fig. 3

two core abstract types, Point and Color. The Point and Color types are then used to define the key types in Skia. An Image is defined as a Color for every Point while Shapes are defined as predicates on Points. Equivalence of two Images is defined as assigning the same Color to each Point. Additionally, since a major focus of this semantics is reasoning about color blending and filtration, types for Blends and Filters are defined as binary and unary operators on colors.

This abstract model hides substantial complexity in the actual implementation.[4] The actual Skia implementation defines Point to be a pair of 32-bit signed integers and Color to be a 32-bit unsigned integer interpreted as ARGB pre-multiplied byte-sized color,[5] and operations on points and colors must thus be anti-aliased and rounded. The exact anti-aliasing and rounding behavior is not guaranteed by Skia and is also exceptionally complex, including hardware dependence in some back-ends; the abstract model allows our semantics to ignore these details. The equivalences proved are thus *up to* anti-aliasing and rounding details (and also assume away issues like coordinate overflow).

Matching the abstract model to Skia would require actually defining the vast array of Images (solid color fills, gradients, image files), Shapes (rectangles, paths, text outlines), Blends (roughly two dozen), and Filters (in fact, these can be defined by shader programs in a custom SkSL shader language). Defining these objects and their equivalences would be a complex undertaking, especially given the pace of change in Skia itself. Luckily, the compiler described in Section 4 does rather superficial reasoning over these objects and only special-cases a handful of them. Our Lean formalization only defines those special cases. The abstract model nonetheless allows for extensibility; for example, this section does not discuss opacity, but our Lean formalization does include it. This extension does not require any changes to the abstract model, only the addition of specific color and blend operators, and thus re-uses the denotational and operations semantics defined in the remainder of this section.

3.2 Skia as Layers

Skia does not permit arbitrary drawing and blending operations; instead, these always happen in a prescribed order. We thus introduce the language of Layers, shown in Section 3.3, which defines the core `Draw` and `BlendLayer` operations in Skia. We focus on these two operations specifically because they are the core target of the optimizing compiler described in **??**. The Layer constructors can be defined in terms of the abstract model, as shown in Figure 3.

[4] In the same way that a traditional programming language semantics might hide the complexity of actually storing closures on the heap.

[5] "ARGB" gives the in-memory byte order of the alpha, red, green, and blue components of a color. The interpretation of these bytes as an unsigned 32-bit integer will depend on the endianness of the platform itself. The colors are premultiplied, meaning that a color with an alpha value of α will have its red, green, and blue components range from 0 to α; this form makes for somewhat more efficient computation.

```
Layer   ::= Empty()
          | Draw(l: Layer, s: Shape,
                 p: Paint, c: Shape)
          | BlendLayer(l_bot: Layer, l_top: Layer,
                       p: Paint)

Paint   ::= Paint(fill: Image, filter: Filter, blend: Blend)

Command c ::= Draw(g: Shape, p: Paint)
            | DrawText(string, p: Paint)
            | Clip(cl: Shape)
            | Save(); c; Restore()
            | SaveLayer(p: Paint); c; Restore()
            | c; c
```

Fig. 4. Layer provides a simple functional description of Skia images.

Skia has two core operations: drawing a shape onto an image, and blending two images (representing two layers) into one. Each operation is parameterized by a Paint object, which indicates the current blend mode, the fill type, and so on; drawing also requires the clip region from the canvas state. The signatures for each operation are defined in Section 3.3 as constructors of a Layer type. Equivalence of two Layer terms is defined by equivalence of their denotations to Images.

The operations can be denoted to the abstract model, as shown in Figure 3. Empty has a straightforward denotation to an all-Transparent image. BlendLayer is likewise straightforward, applying a blend operator, determined by the paint p, point-wise on the two layers being blended. Draw is most complex. Conceptually, it first performs clipping, constructing the intersection $s \cap c$ of the drawn shape and the clip state; then it performs filling, computing the fill color for each point in the shape; and finally it performs blending on the original color and computed fill for each point. The blend is performed even for points outside $s \cap c$; this is critical for, for example, DstIn blending. Note that the equalities over denotations are compositional. That is, if we know that $[\![l]\!] = [\![l]\!]$, then $[\![m]\!] = [\![m[l/l']]\!]$. The fact that denotations are composable means we can use equalities over denotations as rewrites, enabling us to actually use our semantics to verify optimizations.

3.3 μSkia

μSkia is a simplified version of the Skia API. A μSkia program consists of a sequence of commands whose syntax is defined in Figure 4. μSkia includes the critical features described in Section 2, such as the Draw command for drawing, the Clip command for manipulating the clip state, and the Save, SaveLayer, and Restore stack manipulation commands, μSkia diverges from the full Skia API in two main ways. First, the syntax definition is more regular to eliminate unimportant special cases: Save, SaveLayer, and Restore must be properly bracketed, and the `Draw` and `Clip` commands take arbitrary Geometrys, Second, only a small fraction of the API is modeled. Nonetheless, extending μSkia to cover more of the API is straightforward, and our Lean formalization includes additional features such as opacity, styles, and transforms.

μSkia programs manipulate a state Σ:

$$\Sigma : \text{List[Layer]} \times \text{List[Geometry]}$$

The first list in Σ is the stack of layers manipulated by SaveLayer, while the second list is the stack of canvas states manipulated by Save. In this paper, the canvas state is a single Geometry because the only canvas operation is Clip, but in our Lean formalization the canvas state includes additional fields. As is standard, we treat the head of the list as the top of the stack. The initial state is $\langle [\texttt{Empty()}], [\mathcal{U}] \rangle$, an empty layer and the full clip state, each as single-element stacks. A μSkia program then performs a sequence of steps $(c, \Sigma) \longmapsto \Sigma'$, after which the final state consists of a single layer L, which is the output image, and some clip state, which is discarded.

We now describe the core steps in the operational semantics. The simplest is the Draw command, which modifies the top Layer:

$$\frac{}{\langle \texttt{Draw(g, p)}, \langle l :: L, c :: C \rangle \rangle \longmapsto \langle \texttt{Draw(l, g, p, c)} :: L, c :: C \rangle}$$

Note that the Draw operation on the top Layer includes both the Geometry and Paint from the Draw command as well as the current clip from the canvas state:

The Clip command likewise modifies the top canvas state, intersecting it with the new clip geometry:

$$\frac{}{\langle \texttt{Clip(c')}, \langle L, c :: C \rangle \rangle \longmapsto \langle L, c \cap c' :: C \rangle}$$

Note that the full Skia API includes a `SkClipOp` parameter that allows instead taking the difference between the current and new clip; this paper does not show that (straightforward) extension but our Lean formalization does include it.

The Save command duplicates the top canvas state. Then, when its matching Restore is reached, that top canvas state is discarded:

$$\frac{\langle \texttt{cmd}, \langle L, c :: c :: C \rangle \rangle \longmapsto \langle L', c' :: c :: C \rangle}{\langle \texttt{Save(); cmd; Restore()}, \langle L, c :: C \rangle \rangle \longmapsto \langle L', c :: C \rangle}$$

Note that it does not make any changes to the layer state.

The SaveLayer command, meanwhile, allocates a new, empty layer for the top of the layer stack and, when the Restore command is reached, blends that new layer with the one below:

$$\frac{\langle \texttt{cmd}, \langle \texttt{Empty()} :: l :: L, c :: c :: C \rangle \rangle \longmapsto \langle l' :: l :: L, c' :: c :: C \rangle}{\langle \texttt{SaveLayer(p); cmd; Restore()}, \langle l :: L, c :: C \rangle \rangle \longmapsto \langle \texttt{BlendLayer(l, l', p)} :: L, c :: C \rangle}$$

The inference rule for sequential composition is straightforward and, along with various extensions to μSkia, is given in our Lean formalization, which we intend to submit as an artifact.

The μSkia semantics is deterministic, and programs never get stuck. Equivalence of μSkia programs, which we notate $=_{Skia}$, is defined as equivalence of their final images. This equivalence relation is *not* closed under composition or substitution in general, but it is closed under substitution under SaveLayer commands.

3.4 Rewrites in μSkia

This section provides a number of case studies, using the μSkia semantics to prove equivalences between μSkia programs. It demonstrates that the μSkia semantics are sufficiently powerful to define and prove valuable optimizations on real-world Skia programs. Specifically, we collected the Skia programs used by Google Chrome `140.0.7339.16` to render the Alexa Top 100 websites and examined them to find common, suboptimal patterns. In general, the suboptimality comes from using more SaveLayer commands then necessary, and thus both using more GPU memory and also performing more blends than would otherwise be required. To demonstrate the flexibility of the μSkia semantics, this section shows both general-purpose rewrites that apply to a vast range of Skia programs as well as more specific rewrites that we expect to apply only to Skia programs produced by Google Chrome or only to specific web frameworks or even websites.

3.4.1 SrcOver SaveLayers. SrcOver is by far the most common blend mode, corresponding to the normal intuitive meaning of "drawing over" some other image, and SaveLayers with SrcOver blend mode are often unnecessary, suggesting the following idealized equivalence:

```
... // l1                                  ... // l1
SaveLayer(Paint(p, SrcOver));   =Skia     Save();
  ... // l2                                  ... // l2
Restore()                                 Restore()
```

Specifically, this equivalence holds when the commands labeled ℓ_2 all use SrcOver blend mode.[6] Note that this equivalence replaces the SaveLayer command by an equivalent Save command instead of removing it entirely. This is necessary to allow commands in ℓ_2 to change the canvas state. Save is much cheaper to execute than SaveLayer, since it does not require allocating a new GPU surface, so this equivalence still, in practice, significantly cuts computation time.

This equivalence is proven by induction over commands in ℓ_2. It is convenient to assume that sequential composition is left-associated (we do this in our Lean formalization)

Pavel: Do we?

; this means the induction effectively proceeds by "pealing off" the last drawing command in ℓ_2. After expanding out the μSkia semantics, the inductive step requires proving the following equivalence of Layer terms:

```
BlendLayer(                                  Draw(
  l1                                           BlendLayer(l1, l2 p),
  Draw(l2, g, (SrcOver), c),      =Skia        g, p, c)
  (srcover))
```

Note the Draw command uses a SrcOver blend; this is why the commands in ℓ_2 must all use SrcOver blends. Applying this rewrite once "peals" a single Draw command out of the inner SaveLayer. Applying it repeatedly results in the original equivalence of μSkia programs.

This equivalence is, in turn, equivalent to the following equality in our abstract model, which asserts that SrcOver blends are associative:

$$\text{SrcOver}(a, \text{SrcOver}(b, c)) = \text{SrcOver}(\text{SrcOver}(a, b), c)$$

In our Lean formalization, we provide an instantiation of our abstract model using real-valued coordinates and real-valued colors, and prove this equivalence (including in the case of partial transparency). There is also a base case to the recursion, involving a BlendLayer of an Empty Layer. One could optionally prove the case of a BlendLayer of a BlendLayer Layer, but we did not see such Skia programs generated by Chrome.

3.4.2 Gradient Masks. A more specialized rewrite concerns the use of gradients as masks and is common on Russian websites like `mail.ru`, `dzen.ru`, and `yandex.ru`. All three of these sites seem to use a common web framework that seems to result in Chrome generating the following optimizable Skia code:

```
SaveLayer(p);
  Draw(solid fill, srcover);
  SaveLayer(DstIn);                          SaveLayer(p);
    Draw(Gradient(opaque));        =Skia       Draw(solid fill, srcover);
  Restore();                                 Restore();
Restore();
```

Pavel: What is with this example, why is all the syntax fake??

On the left hand side, a gradient is used to draw the mask image. However, with DstIn blending, only the opacity (alpha) of the mask is used, not its color, and in many cases the entirety of the

[6] If we also include opacity, then we must also restrict the savelayer to be opaque. We have formalized this restriction in our mechanization.

gradient is opaque. As a result, the mask operation is entirely superfluous. It is not clear to use why this pattern is used; possibly the gradient could be configured to be partially transparent but isn't on these websites.

No induction is needed in this example, since only a fixed number of Commands are present. Using the μSkia examples, this rewrite unfolds to:

Pavel: Show the Layer formula

This reduces to the following equation in the abstract model:

$$\mathsf{DstIn}(\mathsf{SrcOver}(\mathsf{Transparent}, a), b) = \mathsf{SrcOver}(\mathsf{Transparent}, a)$$

where b is known to be opaque, which it is because all of the stops of the gradient are themselves opaque.[7] Our Lean formalization proves this formula correct for our real-valued instantiation of the abstract model.

3.4.3 Subsuming Filters. SVG are the primary way to draw graphics on the web. The SVG standard has drawing commands just like Skia. Consider the following SVG drawing and its rendered output.

```
<svg width="80" height="30" viewBox="0 0 80 30">
  <defs>
    <radialGradient id="b" ...>...</radialGradient>
    ...
    <pattern id="x" ...>
      <rect width="100" height="100" fill="#7D2AE7"/>
      ...
    </pattern>
  </defs>
  <mask id="a" ...>
    <path fill="#fff" d="M79.444 18.096 ... 69.192 21.888z"/>
  </mask>
  <rect mask="url(#a)" width="100" height="100" fill="url(#x)"/>
</svg>
```

Fig. 5. SVG snippet used in this subsection, shown alongside its rendered result.

This SVG program, in short, fills a rectangle with a complicated gradient. It then masks in the Canva logo onto the gradient. This is a pattern we have seen before in Section 2. Specifically, the mask is implemented using a savelayer with a dstin blend, when the svg is lowered to Skia commands. But, the conversion is not that straightforward. The default masking method for SVG drawings is luminance masking. That is, white pixels(the brightest color) masks in and black pixels(the darkest color) masks out. If we recall dstin blending, dstin blending is opacity based. So we need to convert white pixels into opaque pixels, and black pixels to transparent pixels. Skia does this using a luminance filter.

```
...                                  ...
draw                                 draw
savelayer dstin                      savelayer dstin
savelayer lumafilter  =Skia          draw mask opaque
draw mask white                      restore
restore                              restore
restore
```

[7] For more complicated fills like images, we could pessimistically assume them to not be opaque. In principle, more precise reasoning might be possible, such as reading the Graphics Control Extension block Transparent Color Index for GIF-formatted images.

Now if the mask is simple i.e. we only mask one shape and not complex intersection of many shapes, we should be able to push in filter into the color of the mask. Which is why our rewrite removes the savelayer filter, while also chaing the color of the mask. We can reduce our μSkia rewrite into a layer rewrite, and we have to prove this statement.

$$srcover(l1, luma(srcover(Transparent, fill))) = srcover(l1, srcover(Transparent, luma(fill)))$$

We can prove this equality correct, because we can prove that srcovering a pixel over the transparent pixel is equivalent to just drawing the pixel. One thing we elide, is the case when we are out of the shape we are trying to draw. Then our equation looks kind of like this

$$srcover(l1, luma(srcover(Transparent, Transparent))) = srcover(l1, srcover(Transparent, luma(Transparer$$

So it is important to note that we can only move filters which have the Transparent pixel as the identity pixel. This is true for the luminance filter that Skia emits.

3.4.4 Dstin to Mask. After performing the above rewrite we are left with

```
...
draw
savelayer dstin
draw mask opaque
restore
```

We have already seen this rewrite before in Section 2.

```
...                                ...
draw                               ...
savelayer dstin    =Skia   clip mask
draw mask opaque           draw
restore
```

While we only asserted its correctness in Section 2, we can actually prove it with the semantics now. Let us first reduce this rewrite to μSkia terms.

```
blendlayer(
  l,
  draw(Empty, g, p, c),   =Skia   clipAll(l, g ∩ c)
  dstin
)
```

`clipAll` is a recursive function over layers, which propogates a clip across a sequence of `draws`. Note that we will be referring to `g ∩ c` as `mask` going forward. We define it as such;

```
clipAll(Empty, m)              = Empty
clipAll(Draw(l, g, p, c), m)   = Draw(clipAll(l, m), g, p, c ∩ m)
```

The proof of the rewrite now follows through induction over the layer `l`. For the base case, where `l` is `Empty`, the rewrite reduces to

$$dstin([\![Empty]\!], [\![Draw(...)]\!]) = [\![clipAll(Empty, mask)]\!]$$

If we recall how dstin blending works, dstin blending in a opaque pixel masks in, while a transparent pixel masks out. Now if the pixel we are blending on is Transprent – which $[\![Empty]\!]$ is – the result remains transparent. The other side of the equality also reduces to Transparent by definition of clipAll.

Now for the base case, l = Draw(l', g', p, c'). Which means we need to prove:

$$dstin([\![Draw(l', g', p, c')]\!], [\![Draw(...)]\!]) = [\![clipAll(Draw(l', g', p, c'), mask)]\!]$$

We now perform case analysis on the location of point to be drawn(remember that layer equalities are quantified over points). When the point to be drawn is outside the mask, we get

$$dstin([\![Draw(l', g', p, c')]\!], Transparent) = [\![clipAll(Draw(l', g', p, c'), mask)]\!]$$

As dstin masks out Transparent pixels, and following the defintion of clipAll:

$$Transparent = [\![Draw(clipAll(l'), g', p, c'/\text{cap}mask))]\!]$$

As the point inside is outside the mask, it is outside the clip, hence proving the equality.

Now we turn to the case where the point to be drawn is outside the mask. Looking at the inductive hypothesis.

$$dstin([\![l]\!], Draw(...)) = [\![clipAll(l, mask)]\!]$$

We know that we are masking with a oapque pixel, meaning

$$[\![l]\!] = [\![clipAll(l, mask)]\!]$$

We have to prove:

$$dstin([\![Draw(l', g', p, c')]\!], [\![Draw(...)]\!]) = [\![clipAll(Draw(l', g', p, c'), mask)]\!]$$

Again we are masking with an opaque pixel, so this reduces to

$$[\![Draw(l', g', p, c')]\!] = [\![clipAll(Draw(l', g', p, c'), mask)]\!]$$

By the definition of clipAll we get

$$[\![Draw(l', g', p, c')]\!] = [\![Draw(clipAll(l', mask), g', p, c' \cap \text{mask})]\!]$$

Using the inductive hypothesis and the compositionality of our denotations:

$$[\![Draw(l', g', p, c')]\!] = [\![Draw(l, g', p, c' \cap \text{mask})]\!]$$

as we know that pt is inside the mask, so $c' \cap mask = c'$ finishing the proof.

4 Compiler

In this section we will discuss the implementation of the rewrite rules we have proven in the previous section within Skia. Given that we have shown our rewrites over terms in μSkia, we would like to perform these rewrites within Skia on a similiar representation of commands. As we have discussed before in Section 2, the method calls of a `SkCanvas` object can be record in an in memory buffer of commands called `SkRecord`. `SkRecord` is a dense collection of `SkRecords` objects.[8] A `SkRecords` object, for example `SkRecords::DrawRect`, has a tag to distinguish it from other `SkRecords` objects and also contains all the information that the specific command needs: To continue our example, a `SkRecords::DrawRect` object will contain a reference to a `SkRect` object which define the bounds of the rectangle being drawn and a `SkPaint`. The buffer managed by `SkRecord` is actually an array of pointers to `SkRecords` objects. The allocation of `SkRecords` object is taken care of by a `SkArenaAlloc` arena allocator. Figure 6 illustrates the memory layout. As the `SkRecord` buffer is the most straightforward way to access the Skia API commands before any drawing occurs, all of are rewrites mutate this buffer. `SkRecord` provides two methods to access the underlying `SkRecords` buffer, `visit` and `mutate`. Both methods also need a functor[9] Thes ematchign functors allow us to not only match but do other operations using the record before returning the reference. For the purpose of rewrite matching, we only use the `SkRecords::Is<...>` functor. `Is` is templated over `SkRecords` and all it does is if the `SkRecords` that `Is` is templated over matches the tag of the `SkRecords` at the index we are visiting, we save a reference to the object in the functor. In short, `Is<A>` lets us know if the `SkRecords` object at a certain index is `A`.

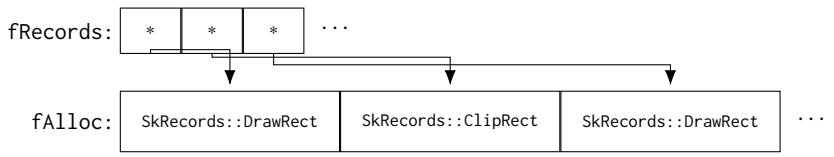


Fig. 6. `SkRecord` stores an array of pointers (`fRecords`) into a contiguous chunk of record objects (`fAlloc`).

Skia provides facilities to serialize the `SkRecord` buffer into a binary format called "Skia Picture" or `skp`. All information in a `SkRecord` format required for drawing, even images drawn, are serialized into a `skp`.[10] Given that `skps` are an easy way to debug and inspect how Chrome generates Skia programs for specific websites, Chrome provides a way to dump `skps` for websites: The `chrome.gpuBenchmarking.printToSkPicture` method dumps `skpss` of a website to disk. We use this facility to collect benchmarks for our evaluation discussed in more detail in Section 5 It is important to note here that Chrome can dump multiple `skps` for one website. Chrome has a compositing step prior to rendering with Skia, wherein it groups different visuals elements of a website into a layers. This is done to reduce the amount of time spent re-rendering when something changes on the website; visual elements that are likely to change together are grouped into the same layer. It is this layer which is converted into a Skia program which the browser finally renders onto the screen. Our optimizer currently only optimizes single layers, and does not perform any cross-layer optimizations. As `skps` can be easily read into and from a `SkRecord` buffer, our optimizer takes as input the `skp` to be optimized, and returns an optimized one.

[8] Confusingly, the collection of the many `SkRecords` object is a uses the singular word `SkRecord`.
[9] Functor here refers to a C++ class that has the function call operator overloaded, not the typeclass implementing an algebraic structure in niche languages like Haskell.
[10] The only exception to what gets serialized seems to be typefaces.

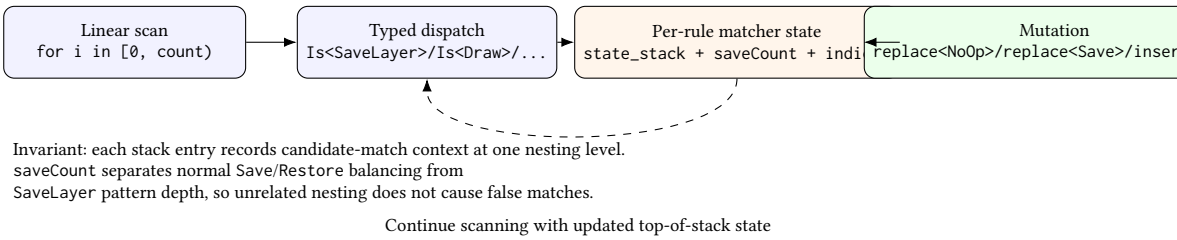


Fig. 7. Implementation-level shape of rewrite matching in `easteregg.cpp`: command dispatch drives a stack matcher, which emits local mutations once a candidate closes at `Restore`.

```
struct RemoveOpaqueSaveLayers {
  void transform(SkRecord* records);

private:
  enum MatchState { Matching, Ignore };
  skia_private::STArray<8, MatchState> state_stack;
  skia_private::STArray<8, int> index_stack;

  SkRecords::Is<SkRecords::SaveLayer> isSaveLayer;
  SkRecords::Is<SkRecords::Save> isSave;
  SkRecords::Is<SkRecords::Restore> isRestore;
  SkRecords::IsSingleDraw isDraw;
};
```

Fig. 8

4.0.1 Mutating `SkRecord`. To perform rewrites over a `SkRecord` buffer, we need to be able to change, remove, and add commands into the buffer. Changing commands is easy. We only need to allocate a new `SkRecords` object in the arena allocator, and change the pointer to point to this new object. Removal is just like changing commands, because we just change the command we want to remove into a special `SkRecords::NoOp` object. Insertion is tricky into a dense array. Compiler IRs usually are represented as doubly linked lists to allow easy constant-time insertion given a location. As `SkRecord` is a dense array, insertion instead takes linear time, because we have to first grow the array to accomadate the new element and then shift all elements after the insertion location to make space for the new element. We can do better by amortizing the cost of many insertions. We do this by storing all the insertions that are to be done. Then when we have determined all the insertions to be done, we allocate space needed for all insertions, and in one linear scan shift all the ncessary elements and perform all insertions. This technique has been given the name "InsertionSet" and is employed in the B3 JIT – which also uses a dense array as its IR memory representation – powering the WebKit browser engine. We have implemented "InsertionSet" in `SkRecord` to facilitate easy insertion of `SkRecords` objects.

4.1 Rewrite Matching

The rewrite we have discussed before in foo are implemented as single-pass matchers over the `SkRecord` command buffer. Each matcher iterates through the command buffer, maintaining the current state of the matcher on a stack. In foo we show the interface of the rewrite matcher.

5 Evaluation

6 Related Work

7 Conclusion